Laporan Tugas Generative AI: Implementasi Conditional GAN Berbasis MLP untuk Generasi Citra Pakaian Fashion-MNIST

Laporan Tugas Mata Kuliah Generative AI

Kelas Generative AI

Disusun oleh:

Ivander Dzaky Khairullah (103012300174),
Muhammad Ragil (103012300015),
Silmi Sila Nia Uma (103012430070)



Program Studi Sarjana Informatika
Fakultas Informatika
Universitas Telkom
Bandung
2026

Dosen Pengampu:
Dr. Aditya Firman Ihsan, S.Si., M.Si.

Abstrak

Penelitian ini mengimplementasikan *conditional Generative Adversarial Network* (cGAN) berbasis *Multi-Layer Perceptron* (MLP) untuk membangkitkan citra *Fashion-MNIST* berukuran 28×28 piksel grayscale pada sepuluh kelas pakaian. Seluruh komponen model dibangun dari komponen dasar PyTorch tanpa memakai model *pretrained* maupun implementasi GAN siap pakai. Generator menerima noise berdimensi 100 dan label *one-hot* berdimensi 10, sedangkan Discriminator menerima citra dan label yang sama untuk menghasilkan satu logit. Training berlangsung 50 epoch pada CPU dengan *BCEWithLogitsLoss*, Adam (learning rate 2×10^{-4}, $\beta = (0{,}5; 0{,}999)$), ukuran *batch* 128, dan seed 42 selama 1.061,2 detik. Kualitas hasil diukur menggunakan classifier MLP yang dilatih pada data asli: akurasi pada data uji asli 0,8755, sementara rata-rata kesesuaian gambar generator terhadap kelas yang diminta 0,8670, dengan kelas terbaik *Trouser* (1,00) dan terlemah *Shirt* (0,62). Hasil ini menunjukkan bahwa model MLP sederhana mampu mempelajari pemetaan noise dan label menjadi bentuk pakaian yang dapat dikenali pada sebagian besar kelas, namun masih menyisakan kekaburan pada kelas yang mirip secara visual seperti *Shirt*, *Coat*, dan *Pullover*. Seluruh kode, pengujian unit (*36 passed*), dan bukti sesi pengerjaan didokumentasikan pada repositori proyek.

Kata Kunci: conditional GAN, multi-layer perceptron, Fashion-MNIST, PyTorch, evaluasi classifier.

Daftar Isi

Abstrak **ii**

Daftar Isi **iii**

Daftar Gambar **v**

Daftar Tabel **vi**

I Pendahuluan **1**
1.1 Latar Belakang 1
1.2 Perumusan Masalah 2
1.3 Tujuan 2
1.4 Hipotesis 2
1.5 Rencana Kegiatan 3
1.6 Jadwal Kegiatan 3

II Kajian Pustaka **5**
2.1 Generative Adversarial Network 5
2.2 Conditional GAN 5
2.3 Multi-Layer Perceptron 6
2.4 Fashion-MNIST 6
2.5 Fungsi Loss dan Optimasi 6
2.6 Evaluasi Model Generatif 6
2.7 Penelitian Terdahulu 6

III Metodologi dan Desain Sistem **8**
3.1 Gambaran Umum Sistem 8
3.2 Kontrak Data dan Preprocessing 9
3.3 Arsitektur Generator 9
3.4 Arsitektur Discriminator 10
3.5 Prosedur Pelatihan 10
3.6 Konfigurasi Eksperimen 11
3.7 Skenario Pengujian 11
3.8 Rancangan Evaluasi 12

IV Hasil dan Pembahasan **13**
4.1 Dataset dan Distribusi Kelas 13
4.2 Hasil Pelatihan 14
4.3 Pengaruh Jumlah Epoch terhadap Kualitas Visual 15
4.4 Sampel Hasil Generator 16
4.5 Keragaman Sampel 18
4.6 Hasil Evaluasi Kuantitatif 18
4.7 Pembahasan 20

V Kesimpulan **21**
5.1 Kesimpulan 21
5.2 Keterbatasan 21
5.3 Saran Pengembangan 21

Daftar Pustaka **23**
A. Struktur Repositori 25
B. Perintah Reproduksi 25
C. Bukti Sesi Pengerjaan 26

Lampiran **25**

Daftar Gambar

3.1 Alur keseluruhan sistem dari data hingga evaluasi 8

4.1 Satu contoh citra asli untuk setiap kelas Fashion-MNIST 13
4.2 Distribusi jumlah sampel per kelas pada data latih 14
4.3 Kurva loss Discriminator dan Generator selama 50 epoch 15
4.4 Perkembangan keluaran Generator pada epoch 1, 25, dan 50 dengan noise tetap 16
4.5 Empat sampel hasil Generator untuk setiap kelas pada epoch 50 17
4.6 Grid satu sampel per kelas dari noise yang sama 18
4.7 Keragaman sampel dalam kelas yang sama (dua belas sampel) . 18
4.8 Akurasi kesesuaian kelas gambar generator dengan garis pembanding akurasi data asli 19

Daftar Tabel

1.1 Pembagian peran anggota kelompok 3
1.2 Jadwal kegiatan pengerjaan 4

3.1 Kontrak antarmuka antar-modul 9
3.2 Konfigurasi lapisan Generator 10
3.3 Konfigurasi lapisan Discriminator 10
3.4 Konfigurasi eksperimen 11

4.1 Ringkasan loss pelatihan pada beberapa epoch 14
4.2 Akurasi classifier pada gambar generator per kelas (100 sampel per kelas) 19

5.1 Struktur direktori repositori 25
5.2 Perintah reproduksi eksperimen 26
5.3 Ringkasan berkas bukti per anggota 26

Bab I

Pendahuluan

1.1 Latar Belakang

Pembangkitan data sintetis menggunakan *Generative Adversarial Network* (GAN) merupakan salah satu pendekatan utama dalam *deep learning* generatif. GAN terdiri atas dua jaringan yang dilatih secara bersamaan: Generator yang berusaha membangkitkan sampel menyerupai data asli, dan Discriminator yang berusaha membedakan sampel asli dari sampel buatan [2]. Bentuk dasar ini tidak memungkinkan pengendalian kelas keluaran. *Conditional GAN* (cGAN) menambahkan informasi kondisi, misalnya label kelas, ke kedua jaringan sehingga Generator dapat diminta menghasilkan sampel pada kelas tertentu [6].

Fashion-MNIST merupakan dataset citra pakaian berukuran 28×28 piksel grayscale dengan sepuluh kelas yang dirancang sebagai pengganti MNIST untuk tugas klasifikasi [10]. Dataset ini banyak dipakai sebagai tolok ukur ringan untuk model generatif karena ukurannya kecil, jumlah kelasnya seimbang, dan bentuk antarkelas cukup beragam meskipun beberapa kelas secara visual mirip, seperti *T-shirt*, *Pullover*, *Coat*, dan *Shirt*.

Sebagian besar implementasi cGAN pada tugas akademik bergantung pada jaringan konvolusional dan model *pretrained* sebagai pembanding kualitas. Pendekatan tersebut memberi hasil visual yang baik, tetapi menyembunyikan mekanisme dasar GAN di balik abstraksi pustaka tingkat tinggi. Pada konteks pembelajaran, dibutuhkan implementasi yang memperlihatkan secara eksplisit bagaimana *conditioning* label, fungsi loss, dan alur optimasi bekerja. Selain itu, penilaian kualitas model generatif tidak dapat hanya mengandalkan nilai loss adversarial karena kedua jaringan saling menyeimbangkan; diperlukan metrik terpisah yang mengukur kesesuaian keluaran dengan kelas yang diminta.

Kesenjangan yang diisi oleh penelitian ini adalah penyediaan cGAN MLP yang dibangun sepenuhnya dari komponen dasar PyTorch, dilengkapi pengujian kontrak (*shape*, rentang nilai, aliran gradien, dan *conditioning*), pipeline data yang reproducible, serta evaluasi kuantitatif memakai classifier yang dilatih dari data asli. Dengan demikian, hasil kerja dapat diverifikasi ulang tanpa bergantung pada model eksternal, dan klaim kualitas didukung angka yang

dapat direproduksi.

1.2 Perumusan Masalah

Berdasarkan latar belakang tersebut, masalah yang dibahas adalah sebagai berikut.

1. Bagaimana merancang Generator dan Discriminator MLP yang mampu memanfaatkan informasi label kelas pada Fashion-MNIST?

2. Bagaimana menyusun pipeline preprocessing, training, dan pembangkitan citra yang konsisten dengan kontrak data rentang $[-1, 1]$ dan dapat direproduksi?

3. Seberapa besar kesesuaian kelas gambar yang dihasilkan generator terhadap label yang diminta, dibandingkan dengan kemampuan classifier yang sama pada data asli?

Ruang lingkup pekerjaan dibatasi pada citra Fashion-MNIST 28×28 grayscale, arsitektur MLP tanpa lapisan konvolusi, pelatihan pada CPU lokal, dan evaluasi menggunakan classifier MLP sederhana. Model *pretrained* dan implementasi GAN siap pakai tidak digunakan.

1.3 Tujuan

Tujuan dari pengerjaan ini adalah:

1. Mengimplementasikan *conditional* GAN berbasis MLP dari komponen dasar PyTorch beserta pengujian kontraknya;

2. Membangun pipeline data, training, dan pembangkitan citra yang reproducible dengan konfigurasi tercatat;

3. Mengukur kesesuaian kelas gambar generator dengan classifier yang dilatih pada data asli dan mengidentifikasi kelas yang paling sulit dipelajari.

1.4 Hipotesis

Model MLP dengan *conditioning one-hot* diperkirakan mampu membangkitkan bentuk pakaian yang dapat dikenali pada kelas-kelas dengan siluet khas, seperti *Trouser*, *Bag*, dan *Sandal*. Untuk kelas yang berbagi siluet serupa, kualitas gambar diperkirakan lebih rendah. Kesesuaian kelas generator diperkirakan berada di bawah akurasi classifier pada data asli karena keterbatasan arsitektur MLP dalam menangkap struktur spasial.

1.5 Rencana Kegiatan

Kegiatan dilaksanakan secara bertahap dan dibagi sesuai peran anggota kelompok: perancangan model dan pengujian kontrak, pipeline data serta training, kemudian evaluasi dan dokumentasi. Rincian peran disajikan pada Tabel 1.1.

Tabel 1.1: Pembagian peran anggota kelompok

No	**Anggota**	**Peran**	**Keluaran utama**
1	Ivander Dzaky Khairullah	*Prompt Engineer* dan arsitek model	`models.py`, `tests/test_models.py`, `architecture.md`
2	Muhammad Ragil	*Data Pipeline* dan *Training*	`data.py`, `train.py`, `configs/default.yaml`, `tests/`
3	Silmi Sila Nia Uma	*Documentation* dan *Evaluasi*	`evaluate.py`, `utils.py`, `docs/`, notebook laporan

1.6 Jadwal Kegiatan

Jadwal pelaksanaan mengikuti alur pengerjaan GitHub per isu dan *pull request*. Tabel 1.2 merangkum urutan pekerjaan beserta artefak yang dihasilkan.

Tabel 1.2: Jadwal kegiatan pengerjaan

No	**Kegiatan**	**Pelaku**	**Artefak**
1	Inisialisasi proyek	Anggota 1	Struktur repositori, `README.md`, `AGENTS.md`
2	Model dan pengujian kontrak	Anggota 1	`models.py`, 15 pengujian model
3	Pipeline data dan training	Anggota 2	`data.py`, `train.py`, pengujian data dan training
4	Evaluasi dan visualisasi	Anggota 3	`evaluate.py`, `utils.py`, notebook laporan
5	Eksperimen dan laporan	Ketiga anggota	Checkpoint, metrik, gambar, laporan

Bab II

Kajian Pustaka

Bab ini membahas dasar teori yang digunakan, yaitu GAN dan cGAN, arsitektur MLP sebagai *generator* dan *discriminator*, dataset Fashion-MNIST, fungsi loss dan optimasi, serta cara evaluasi model generatif. Kajian diarahkan pada pertanyaan penelitian: bagaimana label kelas dapat memengaruhi keluaran generator, dan bagaimana kualitas keluaran tersebut diukur tanpa model *pretrained*.

2.1 Generative Adversarial Network

GAN terdiri atas Generator G dan Discriminator D yang dilatih secara bergantian [2]. Generator memetakan vektor noise z dari distribusi prior p_z menjadi sampel sintetis, sedangkan Discriminator memperkirakan peluang sebuah sampel berasal dari data asli. Fungsi tujuan permainan minmax dirumuskan pada Persamaan (2.1).

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))] \tag{2.1}$$

Dalam praktik, pelatihan Generator menggunakan tujuan non-*saturating* $\max_G \mathbb{E}_z[\log D(G(z))]$ agar gradien tetap kuat saat Discriminator sudah percaya diri. Implementasi pada penelitian ini memakai bentuk loss *binary cross-entropy* dengan logit, yaitu sampel asli diberi target 1 dan sampel palsu diberi target 0.

2.2 Conditional GAN

cGAN menambahkan vektor kondisi y pada masukan kedua jaringan sehingga nilai harapannya menjadi $\mathbb{E}_{x,y}[\log D(x,y)] + \mathbb{E}_{z,y}[\log(1 - D(G(z,y)))]$ [6]. Pada kasus label kelas, y diubah menjadi vektor *one-hot* berdampingan dengan noise pada Generator dan dengan piksel citra pada Discriminator. Mekanisme ini memaksa Discriminator menilai tidak hanya keaslian gambar, tetapi juga konsistensi gambar dengan label yang diberikan.

2.3 Multi-Layer Perceptron

MLP adalah jaringan berlapis penuh yang tersusun dari transformasi linear diikuti fungsi aktivasi non-linear [1]. Lapisan ke-l menghitung $h^{(l)} = f(W^{(l)}h^{(l-1)} + b^{(l)})$. Fungsi aktivasi ReLU dipakai pada Generator, sedangkan Discriminator memakai *LeakyReLU* dengan kemiringan 0,2 agar gradien tetap mengalir untuk nilai masukan negatif.

Pada Generator, lapisan keluaran memakai fungsi *tanh* sehingga nilai piksel berada pada $[-1, 1]$, sesuai normalisasi data. Pada Discriminator, lapisan keluaran berisi satu unit tanpa sigmoid karena loss yang dipakai adalah *BCEWithLogitsLoss*. Pilihan MLP memudahkan inspeksi dan pembuktian aliran gradien, tetapi tidak memanfaatkan kedekatan spasial antarpiksel sebagaimana lapisan konvolusi.

2.4 Fashion-MNIST

Fashion-MNIST berisi 60.000 citra latih dan 10.000 citra uji berdimensi 28×28 grayscale pada sepuluh kelas [10]. Dataset ini dipilih karena seimbang, cepat dilatih pada CPU, dan cukup menantang untuk model generatif sederhana. Kelas yang mirip secara visual, seperti *Shirt*, *Coat*, dan *Pullover*, menjadi titik sulit dalam evaluasi.

2.5 Fungsi Loss dan Optimasi

BCEWithLogitsLoss menggabungkan sigmoid dan *binary cross-entropy* dalam satu operasi yang stabil secara numerik. Untuk logit s dan target $t \in \{0, 1\}$, nilainya dirumuskan pada Persamaan (2.2).

$$\ell(s, t) = \max(s, 0) - st + \log\left(1 + \exp(-|s|)\right) \tag{2.2}$$

Adam mengatur laju belajar per parameter menggunakan estimasi momen pertama dan kedua [5].

2.6 Evaluasi Model Generatif

Metrik seperti *Fréchet Inception Distance* membutuhkan jaringan *pretrained* sehingga tidak sesuai dengan batasan proyek [3]. Alternatif yang tidak memerlukan model eksternal adalah pendekatan dua tingkat: melatih classifier pada data asli, lalu mengukur proporsi gambar generator yang diklasifikasikan sesuai label yang diminta. Metrik ini mengukur kesesuaian kelas, bukan keindahan visual; jika classifier cukup akurat pada data asli, metrik ini memberi gambaran yang mudah ditafsirkan [9].

2.7 Penelitian Terdahulu

Implementasi acuan *DCGAN* menggunakan lapisan konvolusi dan *batch normalization* [4] untuk menghasilkan citra berkualitas lebih tinggi [8]. Penelitian ini tidak berupaya menyaingi hasil tersebut, melainkan menyediakan

baseline MLP yang transparan, mudah diuji, dan seluruh hasilnya dapat direproduksi tanpa model luar.

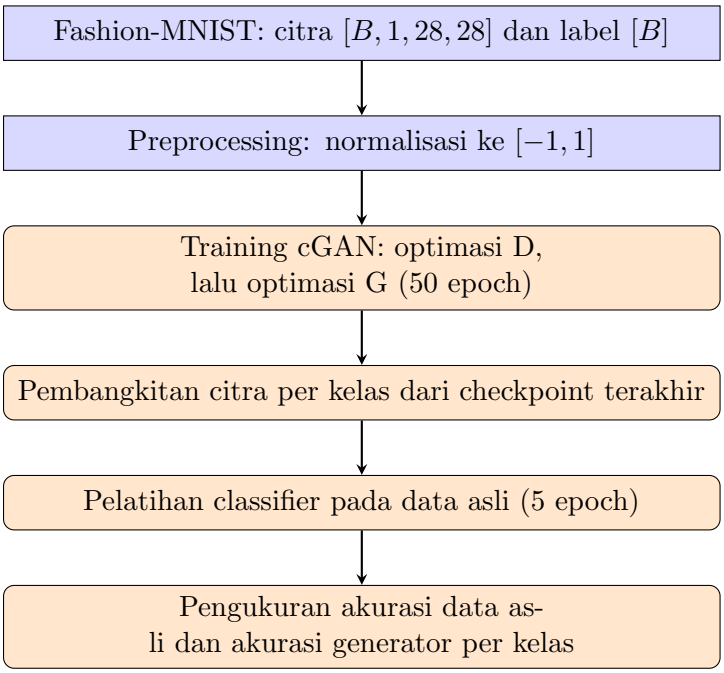
Bab III

Metodologi dan Desain Sistem

Bab ini menjelaskan rancangan sistem, mulai dari alur keseluruhan, kontrak data, arsitektur kedua model, konfigurasi pelatihan, hingga skenario pengujian. Semua tahapan dijalankan secara lokal pada CPU tanpa layanan komputasi awan.

3.1 Gambaran Umum Sistem

Sistem terdiri atas lima tahap berurutan: penyiapan dataset, pelatihan cGAN, pembangkitan citra, pelatihan classifier evaluasi, dan pengukuran kesesuaian kelas. Alur tersebut ditunjukkan pada Gambar 3.1.



Gambar 3.1: Alur keseluruhan sistem dari data hingga evaluasi

3.2 Kontrak Data dan Preprocessing

Dataset dimuat dengan pustaka `torchvision` di atas kerangka PyTorch [7]. Setiap citra dikonversi menjadi tensor float dan dinormalisasi menggunakan rata-rata dan simpangan baku 0,5 sehingga nilainya berada pada $[-1, 1]$, selaras dengan fungsi aktivasi *tanh* pada Generator. Kontrak antar-modul dirangkum pada Tabel 3.1.

Tabel 3.1: Kontrak antarmuka antar-modul

Modul	**Masukan**	**Keluaran**
Dataset	Fashion-MNIST	gambar $[B, 1, 28, 28] \in [-1, 1]$, label $[B]$, kelas 0..9
Generator	noise $[B, 100]$, label $[B]$	gambar $[B, 1, 28, 28]$ dalam $[-1, 1]$
Discriminator	gambar $[B, 1, 28, 28]$, label $[B]$	logit $[B, 1]$ tanpa sigmoid
Training loop	batch gambar dan label	loss D dan loss G per epoch, checkpoint, CSV metrik
Evaluasi	checkpoint, data uji asli	akurasi data asli dan akurasi generator per kelas

Setiap fungsi masukan melakukan validasi eksplisit terhadap bentuk tensor, jumlah sampel, perangkat, tipe data label, dan rentang label. Validasi ini mencegah kesalahan senyap ketika *conditioning* label diabaikan atau saat batch berukuran satu.

3.3 Arsitektur Generator

Generator menerima noise $z \in \mathbb{R}^{100}$ dan label kelas $y \in \{0, \ldots, 9\}$ yang diubah menjadi vektor *one-hot* berdimensi 10. Keduanya digabung menjadi vektor 110 dimensi, lalu diproses melalui dua lapisan *fully connected* berukuran 256 dan 512 dengan aktivasi ReLU, sebelum lapisan keluaran 784 unit dengan aktivasi *tanh*. Keluaran kemudian dibentuk ulang menjadi citra $1 \times 28 \times 28$.

Konfigurasi lapisan ditunjukkan pada Tabel 3.2.

Tabel 3.2: Konfigurasi lapisan Generator

No	**Lapisan**	**Ukuran**	**Aktivasi**
1	*Concat* (noise + label)	110	–
2	*Linear*	256	ReLU
3	*Linear*	512	ReLU
4	*Linear*	784	*tanh*
5	*Reshape*	$1 \times 28 \times 28$	–

3.4 Arsitektur Discriminator

Discriminator meratakan citra menjadi vektor 784 dimensi, menggabungkannya dengan label *one-hot* 10 dimensi, lalu memprosesnya melalui dua lapisan *fully connected* berukuran 512 dan 256 dengan *LeakyReLU* berkemiringan 0,2. Lapisan keluaran menghasilkan satu logit per sampel. Rincian disajikan pada Tabel 3.3.

Tabel 3.3: Konfigurasi lapisan Discriminator

No	**Lapisan**	**Masukan**	**Keluaran**	**Aktivasi**
1	*Concat* flattened image + label	794	794	–
2	*Linear*	794	512	*LeakyReLU*
3	*Linear*	512	256	*LeakyReLU*
4	*Linear*	256	1	–

3.5 Prosedur Pelatihan

Setiap iterasi memperbarui Discriminator terlebih dahulu menggunakan sampel asli berlabel 1 dan sampel palsu berlabel 0, lalu memperbarui Generator dengan tujuan menipu Discriminator pada sampel palsunya. Parameter Discriminator dibekukan selama pembaruan Generator. Pseudokode prosedur ini ditunjukkan pada Algoritma 1.

Algorithm 1 Prosedur pelatihan conditional GAN

procedure TRAINCGAN($T, \{(x_i, y_i)\}, z_{\text{dim}}$)
    Inisialisasi G, D, optimizer Adam O_G, O_D, loss ℓ
    for epoch $= 1$ **to** T **do**
        for setiap batch (x, y) **do**
            $z \sim \mathcal{N}(0, I)$; $\tilde{x} \leftarrow G(z, y)$
            $\mathcal{L}_D \leftarrow \ell(D(x, y), \mathbf{1}) + \ell(D(\tilde{x}, y), \mathbf{0})$
            Perbarui D dengan $\nabla_{\theta_D} \mathcal{L}_D$
            Bekukan parameter D
            $\mathcal{L}_G \leftarrow \ell(D(G(z, y), y), \mathbf{1})$
            Perbarui G dengan $\nabla_{\theta_G} \mathcal{L}_G$
            Aktifkan kembali parameter D
        end for
        Simpan checkpoint epoch dan metrik loss rata-rata
    end for
end procedure

3.6 Konfigurasi Eksperimen

Konfigurasi pelatihan disimpan dalam berkas YAML agar setiap run dapat direproduksi. Nilai yang dipakai dirangkum pada Tabel 3.4.

Tabel 3.4: Konfigurasi eksperimen

Parameter	**Nilai**
Seed	42 (Python, NumPy, PyTorch, urutan *shuffle DataLoader*)
Ukuran batch	128
Jumlah epoch	50
Optimizer	Adam, learning rate 2×10^{-4}, $\beta = (0{,}5; 0{,}999)$
Fungsi loss	`BCEWithLogitsLoss`
Dimensi noise	100
Jumlah kelas	10
Perangkat	CPU
Keluaran	checkpoint per epoch, `metrics.csv`, grid PNG

3.7 Skenario Pengujian

Pengujian dibagi menjadi tiga lapis. Pertama, pengujian unit untuk bentuk keluaran, rentang nilai, aliran gradien, sensitivitas *conditioning*, serta validasi masukan model. Kedua, pengujian kontrak data untuk bentuk, tipe, rentang

normalisasi, dan reproduksibilitas urutan *shuffle*. Ketiga, pengujian integrasi training dan evaluasi yang memastikan checkpoint dapat dimuat, metrik tercatat, serta laporan evaluasi dihasilkan. Semua pengujian dijalankan dengan perintah `pytest tests` menggunakan `PYTHONPATH=src`.

3.8 Rancangan Evaluasi

Classifier evaluasi adalah MLP dengan satu lapisan tersembunyi berukuran 256 dan dilatih dari data asli selama lima epoch, sehingga tidak bergantung pada bobot *pretrained*. Evaluasi menghasilkan dua angka: akurasi pada data uji asli dan akurasi generator dihitung dari 100 gambar per kelas yang dibangkitkan dengan noise ber-seed tetap. Akurasi generator a_c untuk kelas c dihitung dengan Persamaan (3.1).

$$a_c = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}\left[\arg\max_{k} f_k(G(z_i, c)) = c\right] \tag{3.1}$$

dengan $N = 100$ dan f_k adalah logit classifier untuk kelas k. Nilai rata-rata akurasi generator adalah rata-rata a_c atas sepuluh kelas.

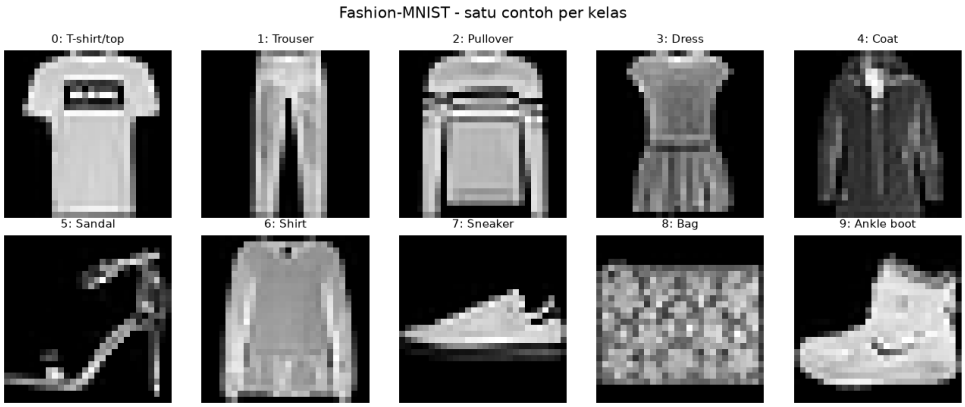
Bab IV

Hasil dan Pembahasan

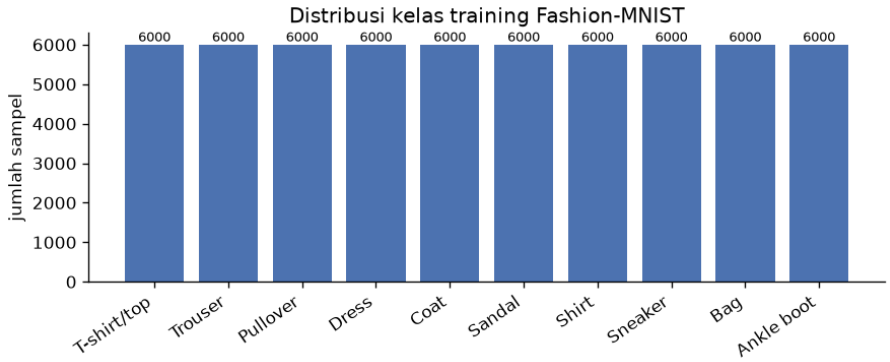
Bab ini menyajikan hasil pelaksanaan eksperimen dan pembahasannya. Seluruh angka berasal dari satu run bersih 50 epoch pada CPU dengan konfigurasi pada Tabel 3.4. Checkpoint, metrik, dan gambar disimpan pada direktori `outputs/` dan tidak disertakan dalam repositori karena berukuran besar.

4.1 Dataset dan Distribusi Kelas

Fashion-MNIST menyediakan 60.000 citra latih dengan distribusi kelas seimbang, yaitu 6.000 citra per kelas. Contoh satu citra per kelas ditunjukkan pada Gambar 4.1 dan distribusinya pada Gambar 4.2. Keseimbangan ini menghindari bias kelas akibat jumlah sampel yang tidak merata, sehingga perbedaan kualitas antarkelas pada hasil generator lebih banyak dipengaruhi oleh kesamaan bentuk, bukan jumlah data.



Gambar 4.1: Satu contoh citra asli untuk setiap kelas Fashion-MNIST



Gambar 4.2: Distribusi jumlah sampel per kelas pada data latih

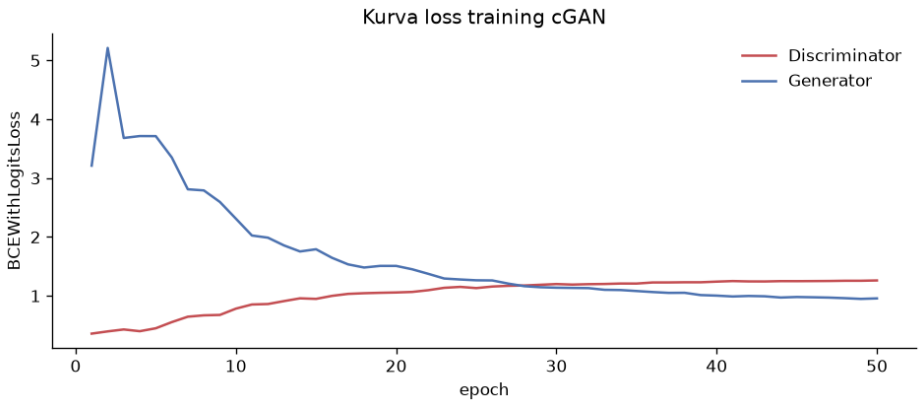
4.2 Hasil Pelatihan

Pelatihan 50 epoch selesai dalam 1.061,2 detik (*wall-clock*) pada CPU tanpa percepatan perangkat keras khusus. Rata-rata loss pada beberapa epoch ditunjukkan pada Tabel 4.1. Nilai diambil dari epoch awal, tengah, dan akhir pelatihan.

Tabel 4.1: Ringkasan loss pelatihan pada beberapa epoch

Epoch	**Loss Discriminator**	**Loss Generator**
1	0,3578	3,2119
3	0,4285	3,6823
25	1,1315	1,2639
49	1,2561	0,9481
50	1,2613	0,9564

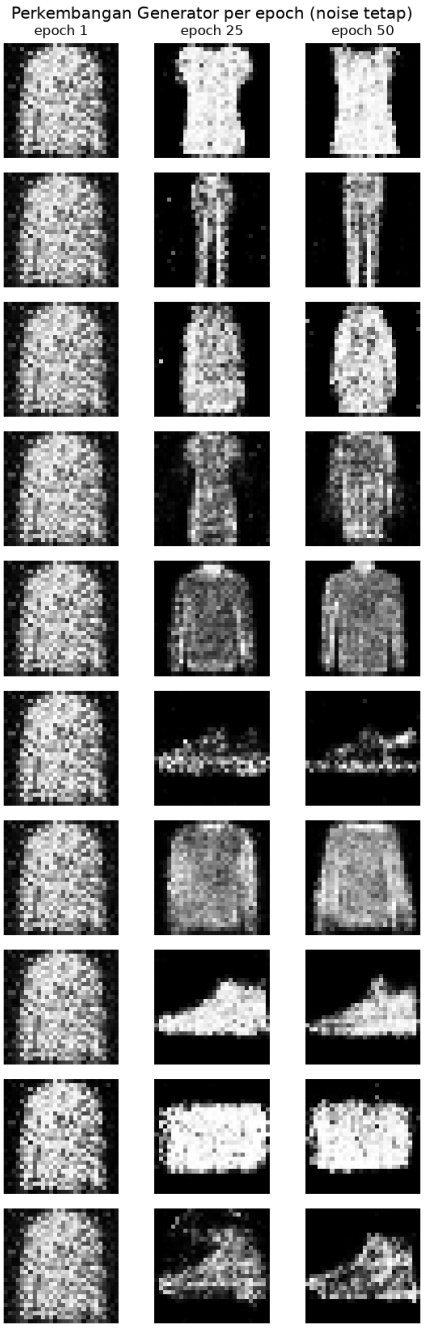
Kurva loss lengkap ditampilkan pada Gambar 4.3. Kedua kurva bergerak saling mendekat dan berosilasi di sekitar nilai 1 pada paruh kedua pelatihan. Pola ini menunjukkan permainan D dan G relatif seimbang, bukan berarti kualitas gambar terus meningkat, karena loss adversarial bersifat relatif terhadap kekuatan lawan.



Gambar 4.3: Kurva loss Discriminator dan Generator selama 50 epoch

4.3 Pengaruh Jumlah Epoch terhadap Kualitas Visual

Gambar 4.4 membandingkan keluaran tiga checkpoint berbeda menggunakan noise dan label yang sama. Perbandingan ini memberi gambaran langsung mengenai perkembangan bentuk, terlepas dari nilai loss. Pada epoch awal, citra masih berupa gumpalan abu-abu tanpa siluet yang jelas. Pada epoch pertengahan, bentuk kaus, tas, dan sepatu mulai muncul. Pada epoch akhir, kontras dan struktur siluet lebih tegas, meskipun masih terdapat kekaburan dan derau di latar.



Gambar 4.4: Perkembangan keluaran Generator pada epoch 1, 25, dan 50 dengan noise tetap

4.4 Sampel Hasil Generator

Gambar 4.5 menyajikan empat sampel berbeda untuk setiap kelas dari checkpoint terakhir. Bentuk yang khas paling jelas terlihat pada kelas *Trouser*,

Bag, *Sneaker*, dan *Ankle boot*. Sebaliknya, *Shirt*, *Pullover*, *Coat*, dan *Dress* cenderung saling menyerupai karena siluetnya beririsan.



Gambar 4.5: Empat sampel hasil Generator untuk setiap kelas pada epoch 50

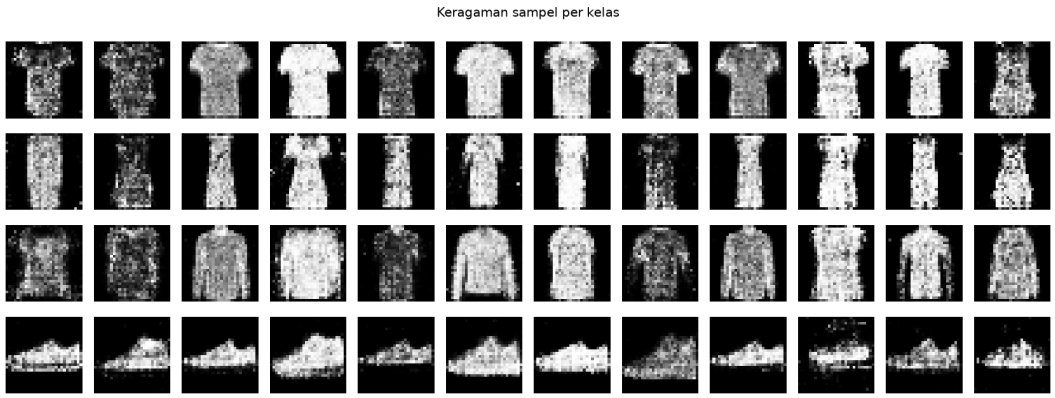
Grid satu gambar per kelas ditunjukkan pada Gambar 4.6. Bentuk grid ini dipakai untuk memeriksa secara cepat apakah *conditioning* label bekerja, karena setiap kolom mewakili label berbeda dari satu vektor noise yang sama.



Gambar 4.6: Grid satu sampel per kelas dari noise yang sama

4.5 Keragaman Sampel

Selain kesesuaian kelas, diperlukan pemeriksaan terhadap *mode collapse*, yaitu kondisi ketika Generator menghasilkan variasi yang hampir sama untuk semua noise. Gambar 4.7 menampilkan dua belas sampel per kelas untuk empat kelas representatif. Variasi bentuk masih terlihat, misalnya perbedaan kemiringan lengan pada kelas *T-shirt* serta variasi sol pada *Sneaker*. Dengan demikian, *mode collapse* total tidak terjadi, meski variasi bentuknya tidak selebar data asli.



Gambar 4.7: Keragaman sampel dalam kelas yang sama (dua belas sampel)

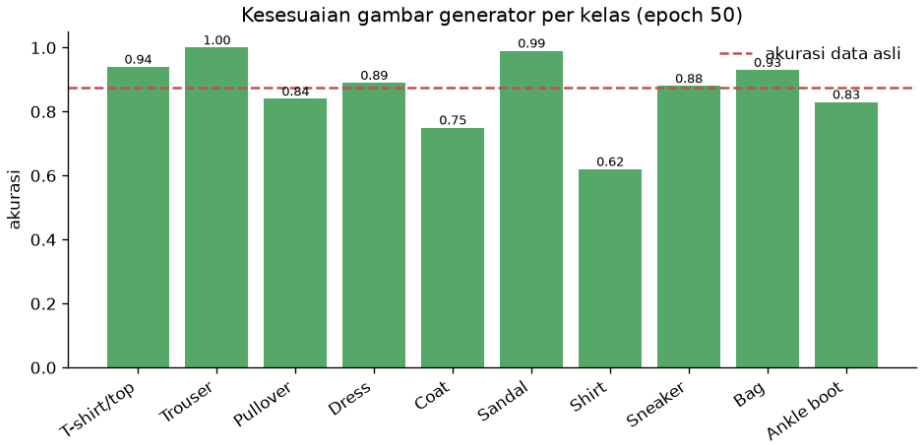
4.6 Hasil Evaluasi Kuantitatif

Classifier MLP dilatih lima epoch pada data asli dan mencapai akurasi 0,8755 pada 10.000 citra uji. Angka ini menjadi pembanding untuk gambar generator. Hasil pada 100 gambar per kelas dirangkum pada Tabel 4.2.

Tabel 4.2: Akurasi classifier pada gambar generator per kelas (100 sampel per kelas)

Label	**Kelas**	**Akurasi**	**Keterangan**
0	*T-shirt/top*	0,94	tinggi
1	*Trouser*	1,00	tertinggi
2	*Pullover*	0,84	sedang
3	*Dress*	0,89	sedang
4	*Coat*	0,75	rendah
5	*Sandal*	0,99	tinggi
6	*Shirt*	0,62	terendah
7	*Sneaker*	0,88	sedang
8	*Bag*	0,93	tinggi
9	*Ankle boot*	0,83	sedang
Rata-rata generator 0,8670 (asli 0,8755)			

Gambar 4.8 memvisualkan perbandingan yang sama dalam bentuk diagram batang. Rata-rata akurasi generator 0,8670 hanya berselisih 0,0085 dari akurasi data asli. Selisih kecil ini perlu ditafsirkan hati-hati. Karena semua kelas pakaian berjumlah seimbang dan classifier cukup akurat, hasil ini menunjukkan bahwa mayoritas gambar generator sudah menyerupai kelas asalnya menurut pandangan classifier. Namun, classifier yang dipakai berkapasitas kecil sehingga belum tentu menangkap detail tekstur dan cacat lokal yang terlihat oleh mata manusia.



Gambar 4.8: Akurasi kesesuaian kelas gambar generator dengan garis pembanding akurasi data asli

4.7 Pembahasan

Tiga temuan utama dapat disimpulkan dari hasil di atas. Pertama, *conditioning* label terbukti berpengaruh: gambar dengan label berbeda memiliki distribusi bentuk yang khas, dan kelas dengan siluet unik memperoleh akurasi tertinggi. Kedua, kelas yang secara visual mirip memperoleh akurasi terendah, yang konsisten dengan keterbatasan MLP dalam menangkap perbedaan halus seperti kerah dan panjang lengan. Ketiga, loss adversarial tidak sejalan lurus dengan kualitas visual; pada paruh kedua pelatihan, loss D justru naik sementara bentuk gambar membaik.

Perbandingan dengan arsitektur konvolusional menunjukkan bahwa DCGAN menghasilkan detail yang lebih tajam karena memanfaatkan struktur spasial [8]. Batasan ini sesuai dengan pilihan metodologis pada Bab III, dan perbaikan berupa lapisan konvolusi dapat diperlakukan sebagai eksperimen lanjutan terpisah tanpa mengubah kontrak data yang sudah ada.

Dari sisi rekayasa perangkat lunak, pemisahan modul menghasilkan pengujian yang dapat dijalankan cepat. Seluruh 36 pengujian lulus pada CPU dalam hitungan detik, termasuk pengujian bahwa perubahan label benar-benar memengaruhi keluaran Generator saat bobot dikendalikan, serta pengujian bahwa resume dari checkpoint melanjutkan penomoran epoch dengan benar.

Bab V

Kesimpulan

5.1 Kesimpulan

Berdasarkan hasil implementasi dan eksperimen, dapat disimpulkan hal berikut.

1. Conditional GAN berbasis MLP berhasil diimplementasikan dari komponen dasar PyTorch dan mampu membangkitkan citra Fashion-MNIST 28×28 pada sepuluh kelas. Label *one-hot* yang digabungkan dengan noise pada Generator serta dengan piksel pada Discriminator terbukti memengaruhi keluaran, baik melalui pengujian unit terkontrol maupun melalui perbedaan bentuk antarkelas pada hasil visual.

2. Pipeline data, training, pembangkitan, dan evaluasi berjalan dengan konfigurasi tercatat dan seed tetap. Pelatihan 50 epoch pada CPU selesai dalam 1.061,2 detik dengan loss akhir Discriminator 1,2613 dan Generator 0,9564. Seluruh 36 pengujian unit lulus.

3. Rata-rata kesesuaian kelas gambar generator mencapai 0,8670 dengan pembanding akurasi classifier pada data asli 0,8755. Kelas bersiluet khas memperoleh akurasi tinggi (*Trouser* 1,00; *Sandal* 0,99), sedangkan kelas yang mirip secara visual memperoleh akurasi rendah (*Shirt* 0,62; *Coat* 0,75).

5.2 Keterbatasan

Arsitektur MLP tidak memanfaatkan struktur spasial sehingga detail tepi dan tekstur pakaian masih kabur. Classifier evaluasi berkapasitas kecil dan dilatih singkat, sehingga angka akurasi bersifat relatif dan tidak mengukur kualitas estetika. Metrik yang dipakai tidak memperhitungkan keragaman secara kuantitatif; pemeriksaan *mode collapse* hanya dilakukan secara visual.

5.3 Saran Pengembangan

Pengembangan yang disarankan adalah mengganti atau menambah lapisan konvolusi (misalnya DCGAN) sebagai eksperimen terpisah, menambah metrik

keragaman yang tetap tanpa model *pretrained*, serta memperbesar kapasitas classifier evaluasi agar penilaian lebih sensitif terhadap cacat lokal.

Daftar Pustaka

[1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[2] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, volume 27, 2014.

[3] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[4] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning*, pages 448–456, 2015.

[5] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[6] Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets. *arXiv preprint arXiv:1411.1784*, 2014.

[7] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[8] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2015.

[9] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training gans. *arXiv preprint arXiv:1606.03498*, 2016.

[10] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

Lampiran

Struktur repositori yang dipakai selama pengerjaan ditunjukkan pada Tabel 5.1. Berkas data, checkpoint, dan hasil eksperimen tidak dimasukkan ke dalam repositori karena ditangani melalui `.gitignore`.

Tabel 5.1: Struktur direktori repositori

Path	**Keterangan**
`src/fashion_cgan/models.py`	Generator dan Discriminator MLP beserta validasi masukan
`src/fashion_cgan/data.py`	Loader Fashion-MNIST dengan normalisasi $[-1, 1]$ dan seed
`src/fashion_cgan/train.py`	Prosedur pelatihan, checkpoint, metrik, opsi resume
`src/fashion_cgan/generate.py`	Pembangkitan grid citra dari checkpoint
`src/fashion_cgan/evaluate.py`	Classifier evaluasi dan pengukuran akurasi per kelas
`src/fashion_cgan/utils.py`	Fungsi bantu denormalisasi, metrik, dan penyimpanan grid
`configs/default.yaml`	Konfigurasi eksperimen
`tests/`	Pengujian unit model, data, training, generate, evaluasi
`notebooks/`	Notebook visualisasi laporan dan bahan presentasi
`docs/`	Arsitektur, rencana implementasi, evaluasi, log sesi AI
`evidence/`	Bukti sesi pengerjaan tiap anggota

Seluruh tahapan dapat dijalankan ulang dengan perintah pada Tabel 5.2. Diasumsikan lingkungan virtual pada direktori `.venv` sudah terpasang melalui `pip install -r requirements.txt`.

Tabel 5.2: Perintah reproduksi eksperimen

Tahap	**Perintah**
Pengujian	`python -m pytest tests -q`
Training	`python -m fashion_cgan.train --device cpu`
Resume	`python -m fashion_cgan.train --resume outputs/cgan_epoch_N.pt`
Pembangkitan	`python -m fashion_cgan.generate outputs/cgan_epoch_50.pt`
Evaluasi	`python -m fashion_cgan.evaluate outputs/cgan_epoch_50.pt`
Visualisasi	Eksekusi `notebooks/report_visuals.ipynb` dari root repositori

Pada Windows PowerShell, variabel `PYTHONPATH` perlu diisi dengan nilai `src` agar paket `fashion_cgan` dapat diimpor.

Bukti pemakaian asisten AI disimpan terpisah per anggota pada direktori `evidence/`. Ringkasannya disajikan pada Tabel 5.3.

Tabel 5.3: Ringkasan berkas bukti per anggota

Anggota	**Berkas bukti**
Ivander Dzaky Khairullah	`session-001-planning.md`, `session-002-generator-discriminator.md`, `session-004-preprocessing-training.md` beserta ekspor JSON asli
Muhammad Ragil	`session-001-data-training-and-github-workflow.md`
Silmi Sila Nia Uma	`session-001-evaluation.md`